ShipPulse

Student Assignment Handout

Beginner-friendly, no-assumption, startup-style lab
Org: AzureDevOpsC07AA | Region: Central India | Date: March 2, 2026

1. What this assignment is

You are joining a startup called ShipPulse. The product has a backend API, a frontend web app, and almost no real DevOps maturity. Your job is to improve it step by step in 7 days.

This lab is written like a guided exercise. The code already exists. You follow exact steps, get exact results, and still have room to improve the approach.

You must use your office laptop limits correctly:

- Local machine: Git and .NET only
- Azure work: Azure Portal, Azure Cloud Shell, GitHub Actions
- No local Azure CLI installation is required

2. Engineer setup model

Both engineers use the same starter code, but each must work in a separate repo inside the same GitHub org.

Engineer	GitHub repo	Resource suffix	Example web app name
Engineer 1	AzureDevOpsC07AA/shippulse-eng1	e1c07aa	app-shippulse-api-dev-e1c07aa
Engineer 2	AzureDevOpsC07AA/shippulse-eng2	e2c07aa	app-shippulse-api-dev-e2c07aa

Each engineer must replace the default suffix `demo01` in the starter repo with their assigned suffix.

3. Architecture you must build

```
Developer Laptop
  -> Git + .NET
  -> GitHub repo (private, inside AzureDevOpsC07AA)
  -> GitHub Actions CI/CD
  -> Azure:
       - Ubuntu VM (dev only) + Nginx
       - Azure App Service (backend API)
       - Azure Static Web Apps (frontend)
       - Azure Key Vault
       - Managed Identity
       - Application Insights + Log Analytics
```

```
- Logic App to start stopped VM
- Optional self-hosted runner on VM
```

4. Starter repo content

The trainer gives you a starter zip. It already contains working code and workflows.

- Backend: .NET 8 Minimal API with /health and /api/feedback
- Frontend: Blazor WebAssembly, built with dotnet only
- Bicep modules for App Service, Key Vault, App Insights, Log Analytics, and optional VM
- GitHub Actions for CI, backend deployment, and frontend deployment

5. Exact naming and region

Use Central India for all resources. If Central India is temporarily blocked by capacity/quota, use South India and mention it in your submission.

Use the exact names below so the trainer can verify quickly.

Resource	Dev name pattern	Prod name pattern	Resource Group	Notes
Resource Group	rg-shippulse-dev	rg-shippulse-prod	same as name	-
App Service Plan	asp-shippulse-dev-<suffix>	asp-shippulse-prod-<suffix>	dev/prod RG	B1
Backend Web App	app-shippulse-api-dev-<suffix>	app-shippulse-api-prod-<suffix>	dev/prod RG	Linux .NET 8
Key Vault	kvshippulsedev<suffix>	kvshippulseprod<suffix>	dev/prod RG	Global unique; letters and numbers only
App Insights	appi-shippulse-dev-<suffix>	appi-shippulse-prod-<suffix>	dev/prod RG	workspace-based
Log Analytics	law-shippulse-dev-<suffix>	law-shippulse-prod-<suffix>	dev/prod RG	30-day retention
Ubuntu VM	vm-shippulse-jump-<suffix>	-	rg-shippulse-dev	dev only
Logic App	la-start-vm-dev-<suffix>	-	rg-shippulse-dev	starts stopped VM
Static Web App	swa-shippulse-dev-<suffix>	swa-shippulse-prod-<suffix>	dev/prod RG	create via Portal

Day 0 - Repo and access setup

Task 0.1 - Create your repo

Engineer 1 creates `AzureDevOpsC07AA/shippulse-eng1`. Engineer 2 creates `AzureDevOpsC07AA/shippulse-eng2`.

Repo requirements:

- Private repo
- Default branches: main and develop
- Branch protection on main: PR required, at least 1 approval
- Branch naming for work: feature/* and hotfix/*

Task 0.2 - Push the starter repo

```
git init
git remote add origin https://github.com/AzureDevOpsC07AA/<your-repo-name>.git
git checkout -b develop
git add .
git commit -m "chore: initial starter"
git push -u origin develop
git checkout -b main
git push -u origin main
```

Expected result: you can see `src/`, `infra/`, and `.github/workflows/` in GitHub.

Task 0.3 - Azure access check

```
az account show
az account list --output table
```

If Azure access fails, stop and ask the trainer before continuing.

Day 1 - Run the app locally and create your first PR

Task 1.1 - Run backend

```
dotnet restore
dotnet run --project src/ShipPulse.Api
```

Expected result: `http://localhost:5080/health` returns status ok.

Task 1.2 - Run frontend

```
dotnet run --project src/ShipPulse.Web
```

Expected result: `http://localhost:5170` opens. Submit feedback. The list refreshes.

Task 1.3 - Make a simple UI change and open a PR

Edit `src/ShipPulse.Web/Shared/MainLayout.razor` and add your engineer label, for example: `Engineer 1`.

Create `feature/day1-ui-change`, push it, and open a PR into `develop`.

Expected result: CI runs on PR. If it fails, fix it and re-run.

Hint: this teaches the repo flow before touching Azure.

Day 2 - Create dev infra, VM, and Nginx

Task 2.1 - Replace the default suffix

Open these files and replace `demo01` with your assigned suffix:

- infra/env/dev.bicepparam
- infra/env/prod.bicepparam

Task 2.2 - Create resource groups

```
az group create -n rg-shippulse-dev -l centralindia
az group create -n rg-shippulse-prod -l centralindia
```

Task 2.3 - Generate SSH key in Cloud Shell

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/shippulse_id_rsa -N ""
cat ~/.ssh/shippulse_id_rsa.pub
```

Copy the public key into `jumpboxSshPublicKey` in `infra/env/dev.bicepparam`.

Set `sshAllowedCidr` to your public IP with `/32`, for example `1.2.3.4/32`.

Task 2.4 - Validate, preview, and deploy dev infra

```
az deployment group validate -g rg-shippulse-dev -f infra/main.bicep -p
infra/env/dev.bicepparam
az deployment group what-if  -g rg-shippulse-dev -f infra/main.bicep -p
infra/env/dev.bicepparam
az deployment group create   -g rg-shippulse-dev -f infra/main.bicep -p
infra/env/dev.bicepparam
```

Expected result: VM, Web App, Key Vault, App Insights, and Log Analytics are created in `rg-shippulse-dev`.

Task 2.5 - SSH to the VM and install Nginx

```
ssh -i <PRIVATE_KEY_PATH> azureuser@<VM_PUBLIC_IP>
sudo apt-get update
sudo apt-get install -y nginx
sudo systemctl enable --now nginx
curl -I http://localhost
```

Expected result: opening `http://<VM_PUBLIC_IP>` shows the Nginx welcome page.

Task 2.6 - Stop the VM manually

Portal -> Virtual Machine -> Stop. You will use Logic App later to start it again.

This directly reuses what you already learned: VM, SSH IP whitelisting, Nginx, port 80, start/stop.

Day 3 - Deploy backend to Azure App Service (Dev)

Task 3.1 - Configure GitHub environment secrets

Create GitHub Environments: `dev` and `prod`.

Add these secrets to the `dev` environment:

- AZURE_CLIENT_ID
- AZURE_TENANT_ID
- AZURE_SUBSCRIPTION_ID
- AZURE_RG_DEV = rg-shippulse-dev
- AZURE_WEBAPP_NAME_DEV = app-shippulse-api-dev-<suffix>

If you already know Service Principal secrets, use them first to get the deployment working. Production-grade upgrade to OIDC comes later.

Task 3.2 - Merge to develop and verify backend deployment

Merge your PR into `develop`. Watch `deploy-backend.yml` in GitHub Actions.

Expected result: `https://<dev-webapp>.azurewebsites.net/health` returns ok.

Day 4 - Deploy frontend to Static Web Apps and create prod infra

Task 4.1 - Deploy prod infra

```
az deployment group validate -g rg-shippulse-prod -f infra/main.bicep -p
infra/env/prod.bicepparam
az deployment group what-if  -g rg-shippulse-prod -f infra/main.bicep -p
infra/env/prod.bicepparam
az deployment group create   -g rg-shippulse-prod -f infra/main.bicep -p
infra/env/prod.bicepparam
```

Task 4.2 - Create Static Web Apps through Portal

Create two Static Web Apps manually through the Portal:

- Dev SWA: `swa-shippulse-dev-<suffix>` connected to branch `develop`
- Prod SWA: `swa-shippulse-prod-<suffix>` connected to branch `main`

Get the deployment token for each SWA and add GitHub secrets:

- AZURE_STATIC_WEB_APPS_API_TOKEN_DEV
- AZURE_STATIC_WEB_APPS_API_TOKEN_PROD
- API_BASE_URL_DEV = https://<dev-webapp>.azurewebsites.net
- API_BASE_URL_PROD = https://<prod-webapp>.azurewebsites.net

Expected result: dev frontend loads from its SWA URL and can submit feedback to the dev backend.

Day 5 - Key Vault and Managed Identity

Task 5.1 - Create the runtime secret

In the dev Key Vault, create a secret:

- Name: `ExternalApi--ApiKey`
- Value: `DEMO-123` (or any sample value)

Task 5.2 - Grant Key Vault access to the Web App

Key Vault -> Access control (IAM) -> Add role assignment -> `Key Vault Secrets User` -> assign to the dev Web App managed identity.

Expected result: the Web App can resolve the Key Vault reference defined in Bicep.

Task 5.3 - Repeat for prod

Create the same secret in prod Key Vault, assign the prod Web App identity, and verify prod deployment still works.

Day 6 - Logic App + Monitoring

Task 6.1 - Create a Logic App to start the stopped VM

Create a Logic App named `la-start-vm-dev-<suffix>` in `rg-shippulse-dev`.

Required flow:

- Trigger: Recurrence
- Time zone: India Standard Time
- For testing: every 5 minutes first
- Action: Start virtual machine `vm-shippulse-jump-<suffix>` in `rg-shippulse-dev`

Test: after the VM is stopped, run the Logic App trigger manually or wait for the interval. Confirm the VM changes to Running.

After testing, change the schedule to a realistic startup schedule such as weekdays 9:00 AM IST.

Task 6.2 - Application Insights

Open the frontend, submit a few feedback items, then verify requests appear in Application Insights.

Task 6.3 - Create one alert

Create a basic alert on the prod Web App for failed requests or 5xx.

Task 6.4 - Cost note

Write a short note: which resource is likely to cost the most, and what you would keep small in a startup.

Day 7 - Production polish

- Ensure prod deploys only from `main`
- Ensure main requires PR and approval
- Write `DECISIONS.md`
- Write `INTERVIEW_NOTES.md`

- Optional: install a self-hosted runner on the dev VM and run one workflow job on it
- Optional: upgrade Azure login from client-secret auth to OIDC

6. Submission checklist

☐ Repo created in AzureDevOpsC07AA

☐ CI passing

☐ Dev VM + Nginx working

☐ Dev backend deployed to App Service

☐ Dev frontend deployed to Static Web Apps

☐ Prod backend deployed

☐ Prod frontend deployed

☐ Key Vault secret working through managed identity

☐ Logic App starts the stopped VM

☐ Application Insights screenshot

☐ Alert screenshot

☐ DECISIONS.md and INTERVIEW_NOTES.md

7. Helpful references

- Azure Cloud Shell features: https://learn.microsoft.com/en-us/azure/cloud-shell/features
- GitHub environments: https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments
- Deploy to App Service from GitHub Actions: https://learn.microsoft.com/en-us/azure/app-service/deploy-github-actions
- App Service Key Vault references: https://learn.microsoft.com/en-us/azure/app-service/app-service-key-vault-references
- Codeless Application Insights for App Service: https://learn.microsoft.com/en-us/azure/azure-monitor/app/codeless-app-service
- GitHub to Azure with OIDC: https://learn.microsoft.com/en-us/azure/developer/github/connect-from-azure-openid-connect